

Let's take a look at an example. The Linux kernel source code is hosted on the git.kernel.org server. To get a copy of this, we need to clone that repository, and what's shown below is the example of this.

```
git clone https://git.kernel.org/cgit/linux/kernel/git/torvalds/linux.git
```

This will create a folder *Linux* in the current directory, which will contain the Linux kernel source code.

Each time you clone, you get a local copy of the repository that is present on the server. Any changes you make will happen to the local copy and not in the server. A copy of the Git repository, which is obtained by cloning, is called the local repository.

Whenever you clone a repository or you initialise a Git repository, all files in the *repo* are called unmodified files. Whenever any change is made to any file, that file is maintained by the Git *repo* as a modified file. If a new file is added to the Git repository, then that file is called an untracked file.

Any changes that are made to the local copy are maintained in the Git repository. Such changes can be viewed by checking the status of the repo. The following is the command to check the status of the Git repository:

```
git status
```

If any file is modified or a new file is added, everything is listed in *Git status*.

Suppose, for instance, in our example, we modify *a.c* and then execute *Git status*, we will get the following output:

```
Changes not staged for commit:
(use "git add <file>..." to update what will be committed)
(use "git checkout -- <file>..." to discard changes in working
directory)
```

```
modified: a.c
```

```
no changes added to commit (use "git add" and/or "git commit
-a")
```

You can see that modified files are listed in the status.

Now, suppose we add a new file called *g.c* in our Git repository and execute *Git status*; you will see the following output:

```
Untracked files:
(use "git add <file>..." to include in what will be committed)
```

```
g.c
```

...where *g.c* is listed in the untracked files section.

These are the changes that will not be present in the Git

server from which this repo was cloned. To put these changes in the Git server, we need to take the following steps:

1. Add the modified files or newly added files to the list of Git files which will get committed.
2. Commit these changes.
3. Push these new commits to the Git server.

So to make Git understand that you want to put these changes in the Git server, it first needs to move the changes from the untracked to the tracked section. To add these changes to the tracked section, the following command is used:

```
git add <file>
```

For our example, we will need to add our modified file *a.c* and newly added file *g.c*; so use the following command:

```
git add a.c g.c
```

After this, if we check the *Git status*, we will see the following output:

```
Changes to be committed:
(use "git reset HEAD <file>..." to unstage)
```

```
new file: g.c
```

```
modified: a.c
```

You can see that under the *Changes to be committed* section, *a.c* and *g.c* are listed. These are the changes that will be committed. To commit these changes, use the command given a little later in this article.

Each set of changes that we make in the repository can be grouped into what is called a Git commit. Each commit in Git has a numeric value, using which it can be reverted to. Whenever you make a set of changes that you think is perfect, you group those changes, and make them a commit before starting off on new changes. Then, while making new changes, if you feel something is broken, you can revert back to the point of your last commit. Also, you can push this commit to the Git server, so that you can access this change from anywhere in the world.

The following command is used to commit to Git:

```
git commit
```

Once *git commit* is executed, a text editor will open up. You will have to write some message, which will provide the details of changes in this commit. Once the file is saved and closed, a commit is created and the Git repository is updated with that commit.

You can see all the commits in the Git repository by executing the following command:

```
git log
```